Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия, Системное и прикладное программное обеспечение

Дисциплина: Системы искусственного интеллекта

Лабораторная работа №1

Linear Regression & Logistic Regression

Выполнил:
Карнажицкий Максим Романович
Группа: P3311

Проверил:
Королева Ю. А.

г. Санкт-Петербург, 2026

Содержание

1. Ссылка на репозиторий с ноутбуком 2
2. Гиперпараметры 3
3. A. Линейная регрессия 4
   3.1. Анализ датасета 4
      3.1.1. Вывод по первичному исследованию 6
   3.2. `preprocessing` 6
      3.2.1. Исключение лишних признаков 6
      3.2.2. Разделение данных 7
      3.2.3. Категориальные признаки 7
   3.3. Проектирование модели 7
   3.4. Анализ результатов 9
      3.4.1. Метрики 9
      3.4.2. `baseline` 10
4. B. Логистическая регрессия 12
   4.1. Анализ датасета 12
      4.1.1. Вывод по первичному исследованию 14
   4.2. `preprocessing` 15
      4.2.1. Разделение данных и преобразование сложных признаков 15
      4.2.2. Обработка пропусков 15
      4.2.3. Категориальные признаки 15
      4.2.4. Масштабирование 16
   4.3. Проектирование модели 16
   4.4. Анализ результатов 17
      4.4.1. Loss-функция 17
      4.4.2. Метрики 18
      4.4.3. Вывод 19
      4.4.4. Majority Baseline 19
5. Вывод 21

1. Ссылка на репозиторий с ноутбуком

Вся работа с довольно подробными (на мой взгляд) пояснениями велась в `.ipynb` ноутбуке.

https://github.com/xtern0o/sii_labs/tree/main/lab1

Отдельно были созданы файлы:

- `lin_reg.py`, содержащий `class LinearRegression`.
- `log_reg.py`, содержащий `class LogisticRegression`

2. Гиперпараметры

Для того, чтобы сделать общий поток действий реитерабельным, нужно задать `SEED` для рандомной генерации – она используется для `shuffle` индексов при распределении данных для `train` и `test` выборок.

```
SEED = 67
TEST_SIZE = 0.2
```

3. А. Линейная регрессия

```
df = pd.read_csv("data/day.csv", parse_dates=["dteday"])

print("Размер:", df.shape)
df.head()
```

Python

```
Размер: (731, 16)
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	2011-01-01	1	0	1	0	6	0	2	0.344167	0.363625	0.805833	0.160446	331	654	985
1	2	2011-01-02	1	0	1	0	0	0	2	0.363478	0.353739	0.696087	0.248539	131	670	801
2	3	2011-01-03	1	0	1	0	1	1	1	0.196364	0.189405	0.437273	0.248309	120	1229	1349
3	4	2011-01-04	1	0	1	0	2	1	1	0.200000	0.212122	0.590435	0.160296	108	1454	1562
4	5	2011-01-05	1	0	1	0	3	1	1	0.226957	0.229270	0.436957	0.186900	82	1518	1600

Рис. 1. Первичное исследование датасета – вывод первых значений

В модели не оказалось дубликатов:

Проверка качества данных и утечки целевой переменной

Проверим дубликаты и утечку данных

```
print("Дубликаты:", df.duplicated().sum())
print("cnt = casual + registered:",
      (df["cnt"] == df["casual"] + df["registered"]).all())
```

Python

```
Дубликаты: 0
cnt = casual + registered: True
```

Рис. 2. Отсутствие дубликатов

3.1. Анализ датасета

Несколько графиков распределения целевого значения `cnt`:

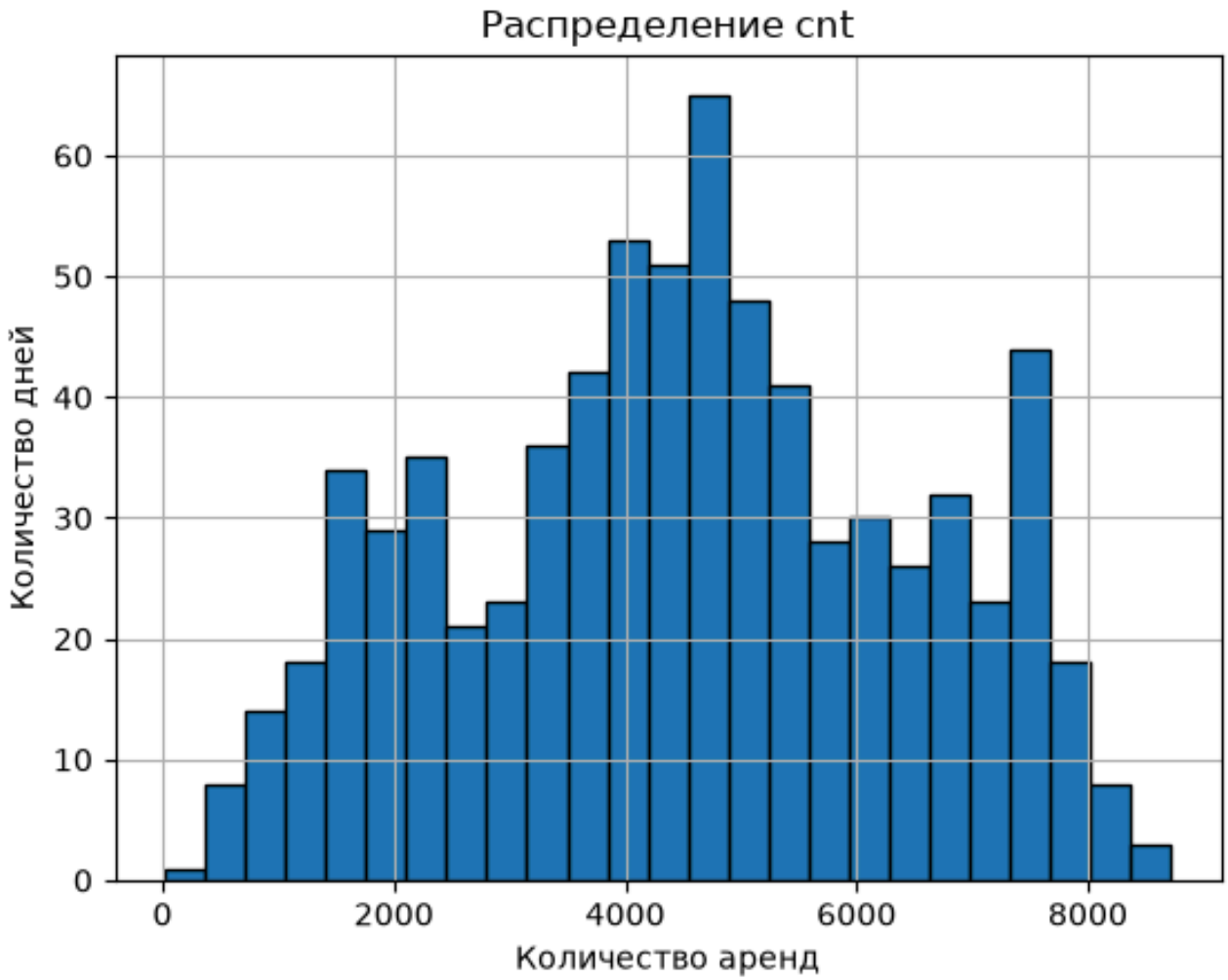


Рис. 3. Распределение занчения `cnt`



Рис. 4. Значения `cnt` в зависимости от текущей даты

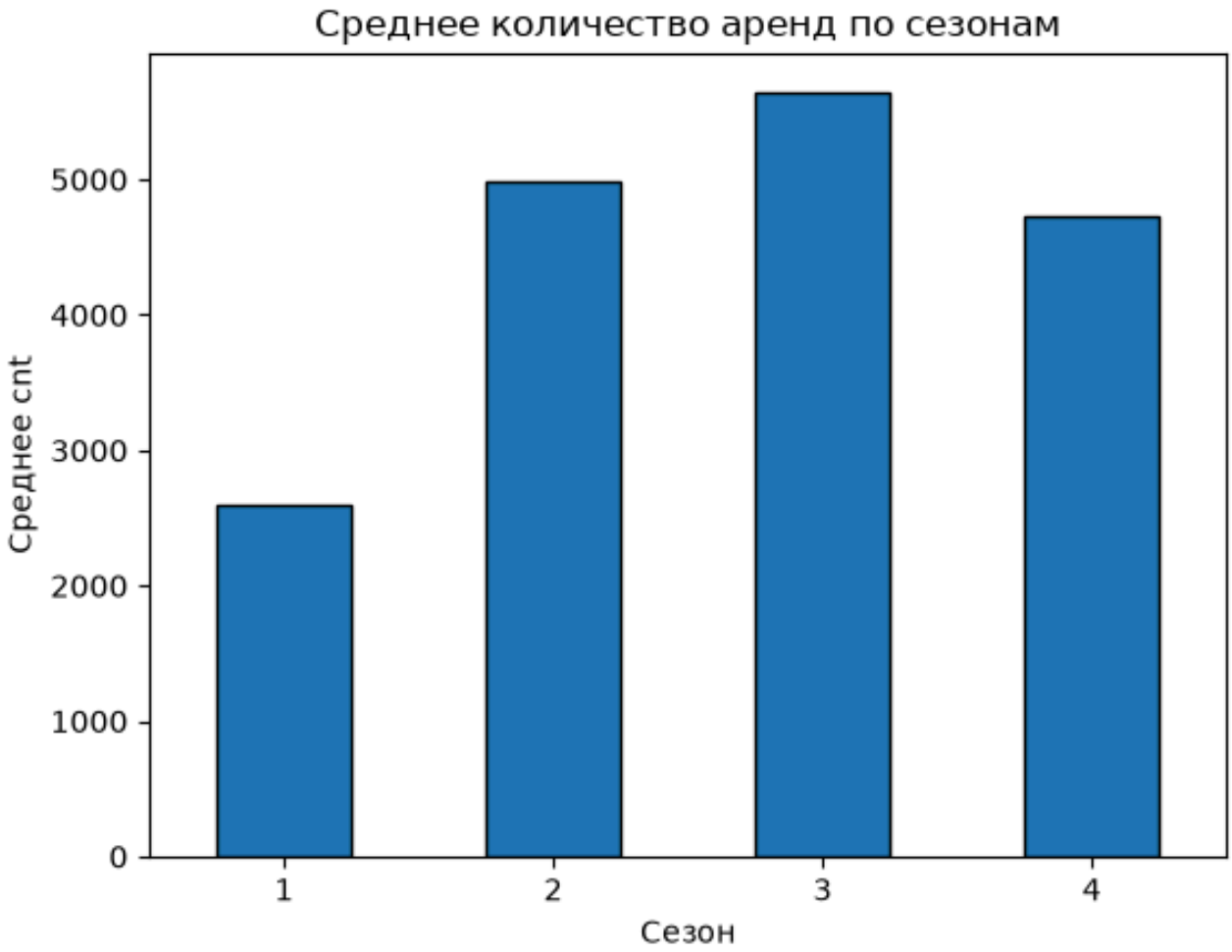


Рис. 5. Распределение `cnt` по сезонам

Глядя на эти графики можно сделать какие-то предположения, но сейчас у нас задачи сделать предположения не стоит.

3.1.1. Вывод по первичному исследованию

- 731 строка и 16 столбцов; данные за 2011-2012 годы
- пропусков и полных дубликатов нет
- $\text{cnt} \in [22; 8714]$; $\overline{\text{cnt}} \approx 4504$.
- $\text{cnt} = \text{casual} + \text{registered}$, поэтому `casual` и `registered` удалим
- `instant` - id строки; его учитывать не будем не будем

3.2. `preprocessing`

3.2.1. Исключение лишних признаков

- создадим целевую переменную `y`, признаки `X`.
- удалим `casual` и `registered`, так как из них напрямую получается `cnt`.
- `instant` и дату тоже использовать не будем

```
y = df["cnt"]
X = df.drop(columns=["cnt", "casual", "registered", "instant",
"dteday"])

X.head()
```

3.2.2. Разделение данных

Теперь разделим данные на `train/test`, воспользовавшись средствами `numpy` и `pandas`

```
indices = np.random.default_rng(SEED).permutation(len(X))
split = int(len(X) * (1 - TEST_SIZE))

train_idx = indices[:split]
test_idx = indices[split:]

X_train = X.iloc[train_idx].copy()
X_test = X.iloc[test_idx].copy()
y_train = y.iloc[train_idx].to_numpy()
y_test = y.iloc[test_idx].to_numpy()
```

3.2.3. Категориальные признаки

- Применим `get_dummies`, чтобы модель не воспринимала номера сезонов, месяцев, дней недели и типов погоды как обычные числа. Каждая категория станет отдельным столбцом из 0 и 1.

```
cat_cols = ["season", "mnth", "weekday", "weathersit"]

X_train = pd.get_dummies(X_train, columns=cat_cols, dtype=int)
X_test = pd.get_dummies(X_test, columns=cat_cols, dtype=int)
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

print("train:", X_train.shape, y_train.shape)
print("test:", X_test.shape, y_test.shape)
X_train.head()
```

[17] Python

```
train: (584, 33) (584,)
test: (147, 33) (147,)
```

	yr	holiday	workingday	temp	atemp	hum	windspeed	season_1	season_2	season_3	...	weekday_0	weekday_1	weekday_2	weekday_3	weekda
412	1	0	1	0.343333	0.351629	0.634583	0.205846	1	0	0	...	0	0	0	0	
52	0	0	1	0.182222	0.186033	0.577778	0.195683	1	0	0	...	0	0	1	0	
374	1	0	1	0.308696	0.318826	0.646522	0.187552	1	0	0	...	0	0	1	0	
695	1	0	1	0.313333	0.339004	0.535417	0.046650	0	0	0	...	0	1	0	0	
663	1	0	1	0.550000	0.529688	0.800417	0.124375	0	0	0	...	0	0	0	0	

5 rows × 33 columns

Рис. 6. Результат разделения данных и `get_dummies` для категориальных признаков

3.3. Проектирование модели

Модель была спроектирвоана в отдельном классе `class LinearRegression`. Методы:

- `__init__(self, learning_rate=0.01, epochs=1000)` – инициализация инстанса и гиперпараметров
- `fit(self, X, y)` – обучение модели с помощью градиантного спуска и лосс-функции `MSE`
- `predict(self, X)` – предсказание нового значения исходя из входных параметров после обучения



Рис. 7.

3.4. Анализ результатов

3.4.1. Метрики

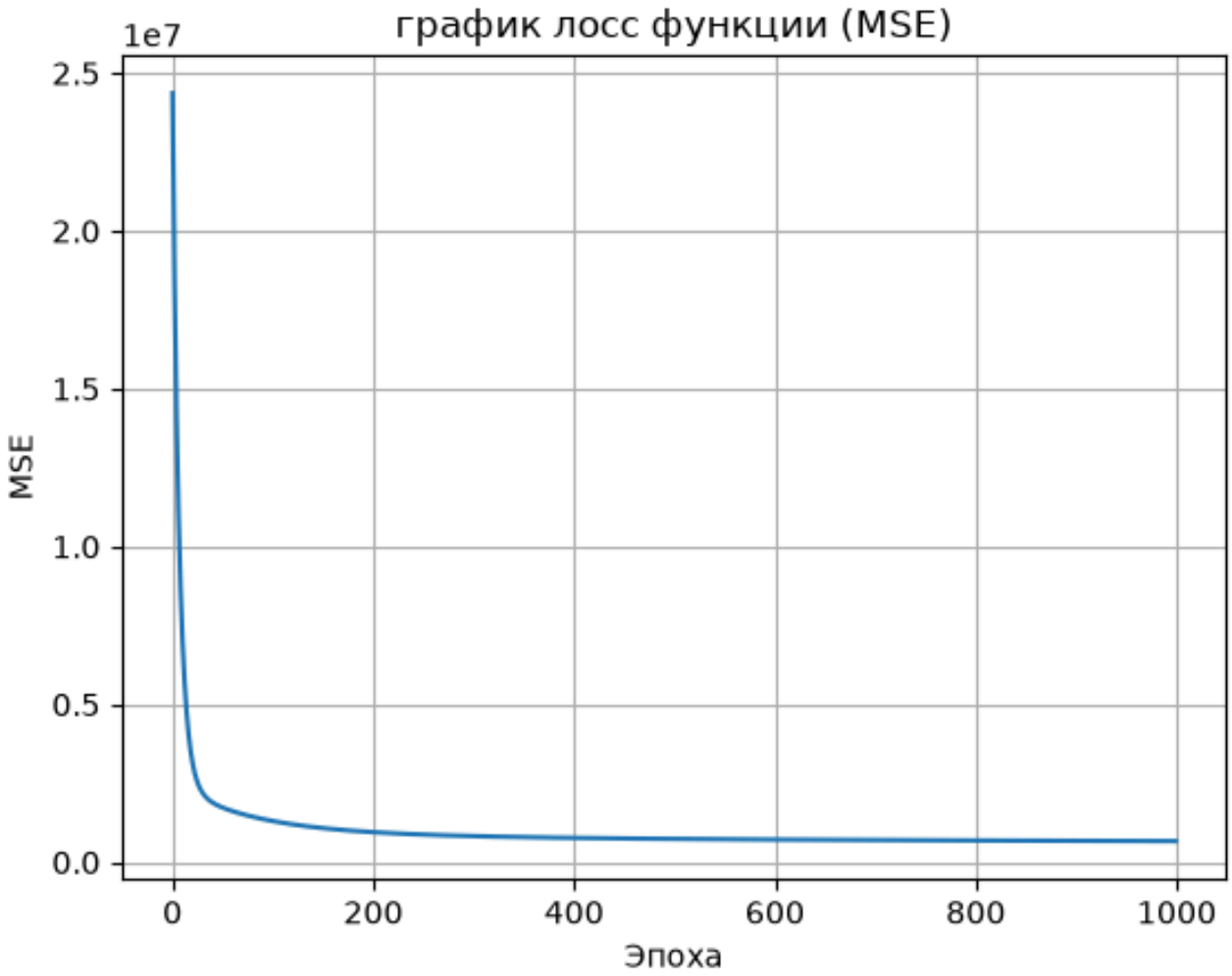


Рис. 8. График лосс-функции обучения модели линейной регрессии

Расчет параметров для оценки модели:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \approx 646.24$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \approx 844.90$$

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \overline{y_i})^2} \approx 0.805$$

```python
y_pred = model.predict(X_test)

mae = np.mean(np.abs(y_test - y_pred))
rmse = np.sqrt(np.mean((y_test - y_pred) ** 2))
r2 = 1 - np.sum((y_test - y_pred) ** 2) / np.sum((y_test - y_test.mean()) ** 2)

print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R^2: {r2:.3f}")
```

Python

```
MAE: 646.24
RMSE: 844.90
R^2: 0.805
```

Рис. 9. Программный расчет праметров

- MAE = 646 означает, что модель в среднем ошибается примерно на 646 аренд
- RMSE = 845 сильнее учитывает большие ошибки. Он выше MAE, значит у модели есть отдельные крупные промахи
- $R^2 = 0.805$ означает, что модель объясняет около 80.5% разброса `cnt`. Значение 1 соответствует идеальной модели, 0 - предсказанию среднего (baseline)

3.4.2. `baseline`

Baseline

в качестве baseline всегда будем предсказывать среднее значение `y_train`. Среднее берём только по train, чтобы не использовать информацию из test.

```python
baseline_pred = np.full(y_test.shape, y_train.mean()) # np array заполненный средним значением по train

baseline_mae = np.mean(np.abs(y_test - baseline_pred))
baseline_rmse = np.sqrt(np.mean((y_test - baseline_pred) ** 2))
baseline_r2 = 1 - np.sum((y_test - baseline_pred) ** 2) / np.sum((y_test - y_test.mean()) ** 2) # очев будет оч близкое к 0

comparison = pd.DataFrame({
    "MAE": [mae, baseline_mae],
    "RMSE": [rmse, baseline_rmse],
    "R^2": [r2, baseline_r2]
}, index=["My Cool LinReg", "Baseline"])

comparison.round(3)
```

Python

Рис. 10. baseline lin reg

	MAE	RMSE	R^2
My Cool LinReg	646.240	844.901	0.805
Baseline	1591.668	1922.266	-0.008

Рис. 11. Сравнение моей модели (`My Cool LinReg`) с `baseline`

- линейная регрессия ошибается в среднем на 646 аренд, baseline - на 1592
- RMSE модели тоже заметно ниже: 845 против 1922

- модель объясняет около 80.5% разброса `cnt`, поэтому она намного лучше простого среднего
- R^2 baseline немного меньше нуля, потому что он использует среднее train, которое отличается от среднего test

Различные графики - см. в ноутбуке

4. B. Логистическая регрессия

4.1. Анализ датасета

```python
import pandas as pd
import matplotlib.pyplot as plt

df_log = pd.read_csv("data/train.csv")

print("size:", df_log.shape)
df_log.head()
```

Python

```
size: (8693, 14)
```

	PassengerId	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck	Name	Transported
0	0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0	0.0	0.0	0.0	0.0	Maham Ofracculy	False
1	0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0	9.0	25.0	549.0	44.0	Juanna Vines	True
2	0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0	3576.0	0.0	6715.0	49.0	Altark Susent	False
3	0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0	1283.0	371.0	3329.0	193.0	Solam Susent	False
4	0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0	70.0	151.0	565.0	2.0	Willy Santantines	True

Рис. 12. Первые несколько значений датасета

```
df_log.info()
df_log.isna().sum().sort_values(ascending=False)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8693 entries, 0 to 8692
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   PassengerId   8693 non-null   str
 1   HomePlanet    8492 non-null   str
 2   CryoSleep     8476 non-null   object
 3   Cabin         8494 non-null   str
 4   Destination   8511 non-null   str
 5   Age           8514 non-null   float64
 6   VIP           8490 non-null   object
 7   RoomService   8512 non-null   float64
 8   FoodCourt     8510 non-null   float64
 9   ShoppingMall  8485 non-null   float64
 10  Spa           8510 non-null   float64
 11  VRDeck        8505 non-null   float64
 12  Name          8493 non-null   str
 13  Transported   8693 non-null   bool
dtypes: bool(1), float64(6), object(2), str(5)
memory usage: 891.5+ KB

CryoSleep       217
ShoppingMall    208
VIP             203
HomePlanet      201
Name            200
Cabin           199
VRDeck          188
Spa             183
FoodCourt       183
Destination     182
RoomService     181
Age             179
PassengerId       0
```

Рис. 13. Пропуски и пустые поля в датасете

Как видно, в датасете есть пустые поля - их нужно чем-нибудь заполнить. Займемся этим в `preprocessing`

Классы в модели сбалансированы почти 50/50 - это хорошо

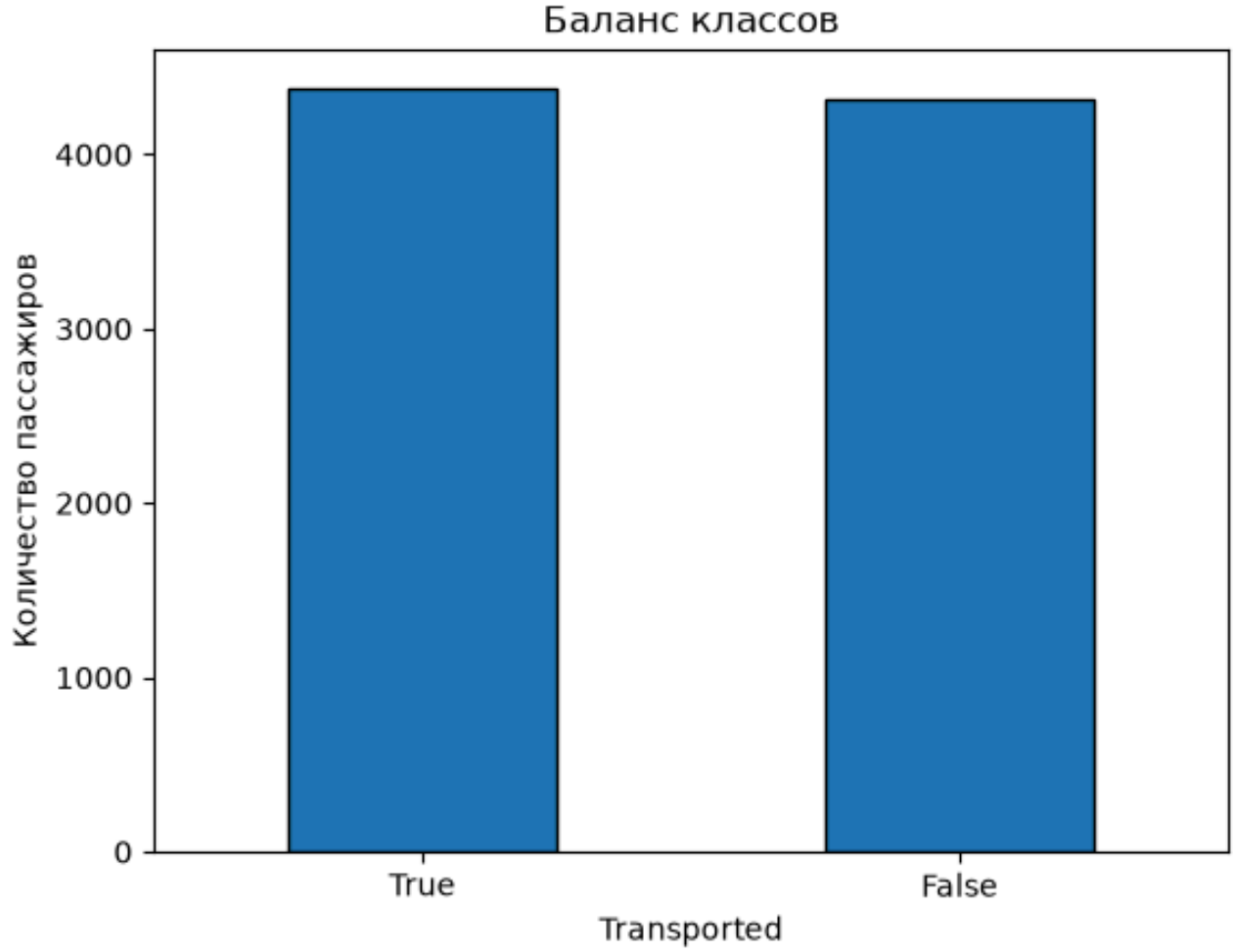


Рис. 14. Диаграмма баланса классов модели

4.1.1. Вывод по первичному исследованию

- в данных 8693 пассажира и 14 столбцов
- целевая переменная почти сбалансирована: 50.4% `True` и 49.6% `False`
- полных дубликатов нет
- пропуски есть почти во всех признаках, позже их нужно заполнить
- расходы сильно скошены: у большинства значения около нуля, но есть очень большие значения
- категориальные признаки позже закодируем через `get_dummies`

4.2. `preprocessing`

4.2.1. Разделение данных и преобразование сложных признаков

1. `Transported` запишем в y, остальные признаки - в X. `Name` и `PassengerId` уберём: это почти уникальные идентификаторы.
2. `Cabin` кстати имеет вид `палуба/номер/сторона`, поэтому разделим его на три отдельных признака.
3. Разделим на train и test в отношении 80%/20%

4.2.2. Обработка пропусков

Числовые пропуски заполним медианой, категориальные — модой. Всё считаем только по `train`, чтобы не тащить инфу из `test`.

```
num_cols = X_train.select_dtypes(include="number").columns
cat_cols = X_train.select_dtypes(exclude="number").columns

for col in num_cols:
    median = X_train[col].median()
    X_train[col] = X_train[col].fillna(median)
    X_test[col] = X_test[col].fillna(median)

for col in cat_cols:
    mode = X_train[col].mode()[0]
    X_train[col] = X_train[col].fillna(mode)
    X_test[col] = X_test[col].fillna(mode)
```

4.2.3. Категориальные признаки

Применим get_dummies к HomePlanet, Destination, Deck и Side. После кодирования получилось 27 признаков.

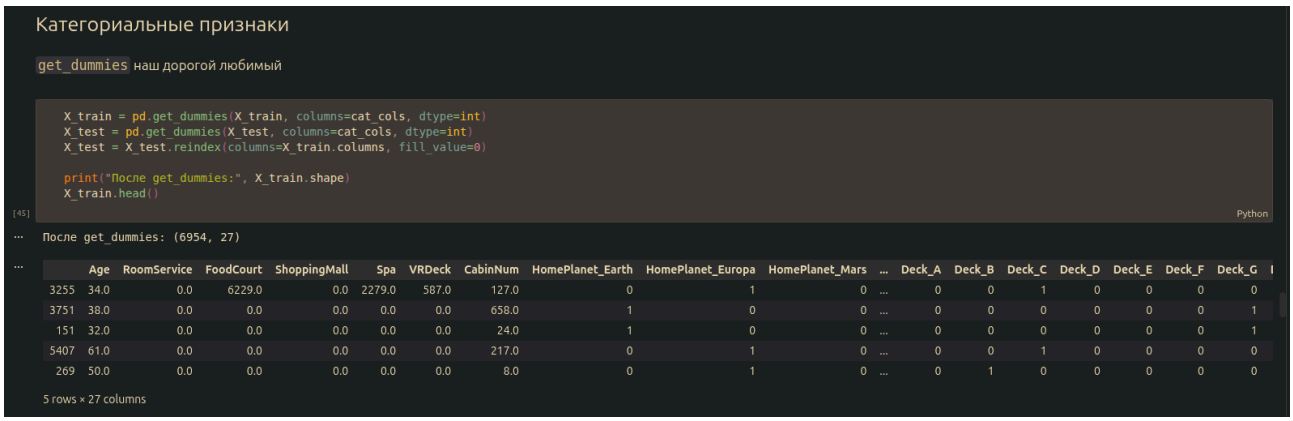


Рис. 15. Результат `get_dummies` для категориальных признаков

4.2.4. Масштабирование

Возраст и траты в разных масштабах, поэтому приведём числовые признаки к среднему 0 и std 1. Параметры опять считаем только по `train`.

4.3. Проектирование модели

Модель спроектирована в отдельном классе `class LogisticRegression`. Методы:

- `__init__(self, learning_rate=0.1, epochs=1000, threshold=0.5)`
- `_sigmoid(self, z)` - вычисление сигмоиды с ограничениями для страховки от переполнения
- `fit(self, X, y)` - обучение модели лог. регрессии с градиентным спуском на лосс-функции `BCE`
- `predict_proba(self, X)` – предсказание вероятности принадлежности сущности к классу 1
- `predict(self, X)` – вывод о принадлежности сущности с учетом порога

Подробный ход мыли о принципе работы – в ноутбуке.



Рис. 16. результаты обучение модели логистической регрессии

4.4. Анализ результатов

4.4.1. Loss-функция

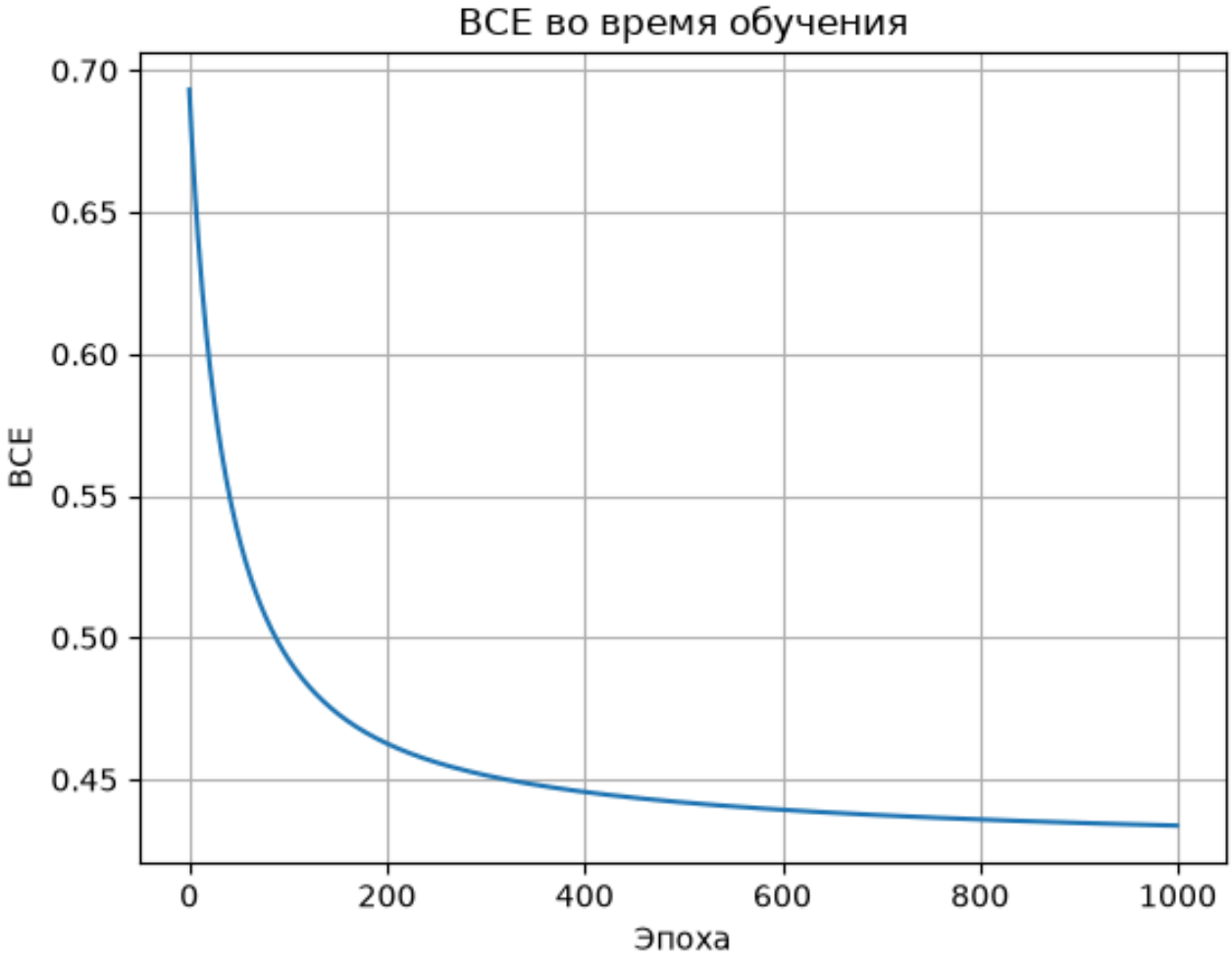


Рис. 17. loss-функция эпох обучения LogReg

4.4.2. Метрики

Метрики на test

```
y_proba = model.predict_proba(X_test)
y_pred = model.predict(X_test)

tp = np.sum((y_test == 1) & (y_pred == 1))
tn = np.sum((y_test == 0) & (y_pred == 0))
fp = np.sum((y_test == 0) & (y_pred == 1))
fn = np.sum((y_test == 1) & (y_pred == 0))

accuracy = (tp + tn) / len(y_test)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)

safe_proba = np.clip(y_proba, 1e-15, 1 - 1e-15)
log_loss = -np.mean(y_test * np.log(safe_proba) + (1 - y_test) * np.log(1 - safe_proba))

metrics = pd.Series({
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1": f1,
    "LogLoss": log_loss
})
metrics.round(3)
```

```
Accuracy     0.771
Precision    0.771
Recall       0.778
F1           0.775
LogLoss      0.462
dtype: float64
```

Рис. 18. ПОлученные метрики

```
confusion_matrix = pd.DataFrame(
    [[tn, fp], [fn, tp]],
    index=["really 0", "really 1"],
    columns=["predicted 0", "predicted 1"]
)
confusion_matrix
```

55]

	predicted 0	predicted 1
really 0	656	203
really 1	195	685

Рис. 19. confusion-matrix

4.4.3. Вывод

- `Accuracy = 0.771` - модель правильно классифицировала 77.1% пассажиров
- `Precision = 0.771` - среди предсказанных `Transported = 1` верными оказались 77.1%
- `Recall = 0.778` - модель нашла 77.8% реально перемещённых пассажиров
- `F1 = 0.775` - precision и recall близки, сильного перекоса между ними нет
- `LogLoss = 0.462` - чем он меньше, тем точнее предсказанные вероятности

для сравнения, если всегда предсказывать вероятность $p = 0.5$, то при любом правильном классе ошибка одного объекта равна:

$$-\ln(0.5) = \ln(2) \approx 0.693$$

поэтому `0.462 < 0.693` означает, что вероятности нашей модели информативнее постоянного ответа 50 на 50.

из 1739 пассажиров модель ошиблась на 398. Ошибки двух типов почти равны: `FP = 203`, `FN = 195`, поэтому модель не имеет заметного перекоса в один класс. BCE на test немного выше, чем на train (`0.462` против `0.434`) - это норм разница, сильного переобучения не видно.

4.4.4. Majority Baseline

Была построена модель majority baseline. Вот сравнение моей модели с ней. Моя лучше.

	Accuracy	Precision	Recall	F1	LogLoss
Моя LogReg	0.771	0.771	0.778	0.775	0.462
Majority baseline	0.506	0.506	1.000	0.672	0.693

Рис. 20. Majority Baseline comparison

5. Вывод

В ходе выполнения лабы я вспомнил, как делается градиентный спуск, изучил основы ML, построил 2 модели для решения задач предсказания и бинарной классификации.